模板骨架 Template/ihxnan.hpp

> **用法**：开题时把本文件整体粘贴到提交文件最上方，在 solve() 内写题解。
>
> - 默认单组用例（t = 1），多组用例时在 solve() 开头读入 t 即可（main 里 while (t--)）。
> - cin / cout 已重定向到快读快写 fin / fout；**仅支持整数、char、string**（double 等类型编译期报错，与提交模式一致，避免"本地能编、评测报错"）。
> - 本地调试编译需 -std=c++20 -DIHXNAN（dbg.hpp 依赖 C++20 source_location），提供 gdb() 变量打印、gdbif() 条件打印、gdb_assert() 断言、TIME() 计时；提交时不加该宏，仅 gdb() / TIME() 退化为空操作 —— **gdbif() / gdb_assert() 在提交版会编译错误**，需删除或包进 #ifdef IHXNAN。
> - 快写基于缓冲区，程序结束由析构统一 flush，勿在中间混用 printf。
> - rd0(x) 一行整体读入容器（vector / array / deque 等，不含 map 嵌套），rd1(x) 从 x[1] 读到末尾（适配 1-indexed 容器）；分块宏 bl / bi / br 需先定义 blen（br 还需 n），fid / gx / gy 需先定义 m。
> - iINF / mod / lINF / MOD 均为 const ll，**不可再赋值**（双模数等场景请自定义变量）。
> - fin 读到 EOF 后 ok 置 false，可用 if (fin) 判断输入是否结束（适合未给组数的多组输入）。

```
#include <bits/stdc++.h>

// ============ 常用宏 ============
#define endl '\n'                               // 输出换行（配合下方 IO 结构体，末尾无多余空白）
#define ls(x) ((x) << 1)                        // 线段树/树状结构：节点 x 的左儿子下标（x*2）
#define rs(x) ((x) << 1 | 1)                    // 线段树/树状结构：节点 x 的右儿子下标（x*2+1）
#define bl(x) ((x) * blen + 1)                  // 分块：块 x 左边界下标（1 起始，含；需已定义 blen）
#define bi(x) (((x) - 1) / blen)                // 分块：下标 x 所属块编号（0 起始；需已定义 blen）
#define br(x) (min(((x) + 1) * blen, n))        // 分块：块 x 右边界下标（受全局 n 限制，含；需已定义 bl
en、n）
#define fid(x, y) ((m) * (x - 1) + (y))         // 二维坐标 (x,y) → 一维编号（1-indexed；需已定义 m）
#define gx(id) (((id) - 1) / m + 1)             // fid 逆变换：编号 → 行号（需已定义 m）
#define gy(id) (((id) - 1) % m + 1)             // fid 逆变换：编号 → 列号（需已定义 m）
#define rd0(x) for (auto &p : x) cin >> p       // 容器整体读入（vector/array/deque 等，不含 map 嵌套）
#define rd1(x) for (int i = 1; i < x.size(); ++i) cin >> x[i] // 从 x[1] 读到末尾（1-indexed 容器，跳过
x[0]）

// ============ 主框架 ============
// t = 用例数（默认 1，多组题在 solve() 开头 cin >> t）；solve() 由选手实现
int t = 1; void solve();
int main() { while (t--) solve(); return 0; } // 循环 t 次调用 solve()

// ============ 类型别名 ============
using namespace std;
using ll = long long;                   // 64 位有符号整数（默认整型）
using ul = unsigned long long;          // 64 位无符号整数（自然溢出哈希等）
using lll = __int128;                   // 128 位整数（GCC 扩展；打印需 -DIHXNAN 的 dbg.hpp）
using ld = long double;                 // 高精度浮点（快读快写不支持，避免 cout）
using vb = vector<bool>;                // bool 向量（位压缩，取址行为特殊）
using vi = vector<int>;                 // int 向量
using vvi = vector<vi>;                 // 二维 int 向量
using vl = vector<ll>;                  // ll 向量
using vvl = vector<vl>;                 // 二维 ll 向量
using pii = pair<int, int>;             // int 二元组
using pll = pair<ll, ll>;               // ll 二元组
using ti = tuple<int, int, int>;        // int 三元组（如查询 (l, r, id)）
using tl = tuple<ll, ll, ll>;           // ll 三元组

// ============ 常量（const ll，不可再赋值）============
const ll iINF = 0x3f3f3f3f,             // int 上无穷大（≈1.06e9，相加不易溢出）
         mod = 998244353,               // 常用 NTT 模数
         lINF = 0x3f3f3f3f3f3f3f3f,     // ll 上无穷大（≈4.56e18，相加不易溢出）
```

```
        MOD = 1000000007;            // 常用质数模数

#ifdef IHXNAN
    // 本地调试模式：dbg.hpp 提供同逻辑 IO 与调试宏（需先 symlink；编译加 -std=c++20）
    #include <dbg>
#else
    // 提交模式：cin/cout 重定向到快读快写；调试宏仅 gdb/TIME 有空操作回退
    #define cin fin
    #define cout fout
    #define gdb(...)   ((void)0)
    #define TIME(name) ((void)0)

    struct IO {
        static const int S = 1 << 21; // 输入/输出缓冲区大小
        uint8_t buf[S], *p1, *p2;
        int st[105], Top;             // st 为输出数位栈
        bool ok;                      // 读到 EOF 后置 false，可用 if (fin) 判断
        IO() : p1(nullptr), p2(nullptr), Top(0), ok(true) { st[0] = 0; }
        ~IO() { clear(); }
        inline void clear() { fwrite(buf, 1, Top, stdout); Top = 0; }

        // 读一个字节（缓冲区空则批量 fread）
        inline int inc() {
            if (p1 == p2) {
                p2 = (p1 = buf) + fread(buf, 1, S, stdin);
                if (p1 == p2) return EOF;
            }
            return *p1++;
        }
        // 写一个字节（缓冲区满则落盘）
        inline void outc(int c) {
            if (Top == S) clear();
            buf[Top++] = (uint8_t)c;
        }
        inline explicit operator bool() const { return ok; }

        // 输出整数（仅算术类型）
        template <typename T>
        inline IO &operator<<(T x) {
            int f = 0;
            if constexpr (std::is_signed_v<T>) { if (x < 0) outc('-'), f = 1; }
            do { st[++st[0]] = f ? -(x % 10) : x % 10, x /= 10; } while (x);
            while (st[0]) outc(48 ^ st[st[0]--]);
            return *this;
        }
        // 输出字符 / 字符串
        inline IO &operator<<(char c) { outc(c); return *this; }
        inline IO &operator<<(const char *x) { while (*x) outc(*x++); return *this; }
        inline IO &operator<<(char *x) { while (*x) outc(*x++); return *this; }
        inline IO &operator<<(const string &x) { for (auto c : x) outc(c); return *this; }

        // 读入整数（跳过非数字，支持负号；EOF 置 ok=false）
        template <typename T>
        inline IO &operator>>(T &x) {
            x = 0;
            int f = 0, ch = inc();
            while (ch != EOF && !isdigit(ch)) { if (ch == '-') f ^= 1; ch = inc(); }
            if (ch == EOF) { ok = false; return *this; }
            while (ch != EOF && isdigit(ch)) x = (x << 3) + (x << 1) + (ch ^ 48), ch = inc();
            f ? x = -x : 0;
            return *this;
        }
        // 读入单个字符（跳过空白）
        inline IO &operator>>(char &x) {
```

```
            int c = inc();
            while (c != EOF && isspace(c)) c = inc();
            if (c == EOF) { ok = false; return *this; }
            x = (char)c;
            return *this;
        }
        // 读入空白分隔的字符串
        inline IO &operator>>(string &x) {
            x.clear();
            int c = inc();
            while (c != EOF && isspace(c)) c = inc();
            if (c == EOF) { ok = false; return *this; }
            while (c != EOF && !isspace(c)) x += char(c), c = inc();
            return *this;
        }
    } fin, fout;
#endif
```

精简版骨架 Template/contest.hpp

> **用法**：正式赛打印后手抄用，比 ihxnan.hpp 短很多：快读改用 ios::sync_with_stdio(false) + cin.tie(nullptr)，调试为内联的简易 gdb，不再 include Debug/dbg.hpp。
>
> - **默认多组用例**：main 内 int t = 1; cin >> t; 无条件读入组数；单组题请注释掉 cin >> t; 。
> - 调试编译 g++ -std=c++17 -O2 -DIHXNAN；提交 g++ -std=c++17 -O2（不加宏时 gdb(...) 展开为 void() ）。
> - gdb() 只支持有 operator<< 的类型（整数、char、string 等）；**vector/map/set/pair/ __int128 等无法直接打印**，变量名含逗号（如 pair<int,int>、f(1,2) ）会被切错。
> - 保留宏 ls / rs / bl / bi / br / fid / gx / gy；未提供 rd0 / rd1 。

```
#include <bits/stdc++.h>

#define endl '\n'                               // 输出换行，配合下方 IO 结构体使用（末尾无多余空白）
#define ls(x) ((x) << 1)                        // 线段树/树状结构：节点 x 的左儿子下标（x*2）
#define rs(x) ((x) << 1 | 1)                    // 线段树/树状结构：节点 x 的右儿子下标（x*2+1）
#define bl(x) ((x) * blen + 1)                  // 分块：块 x 的左边界下标（1 起始，含）
#define bi(x) (((x) - 1) / blen)                // 分块：下标 x 所属的块编号（0 起始，blen 为块长）
#define br(x) (min(((x) + 1) * blen, n))        // 分块：块 x 的右边界下标（受全局 n 限制，含）
#define fid(x,y) ((m)*(x-1)+(y))                // 二维坐标 (x,y) → 一维编号（1-indexed，m 为列数）
#define gx(id) (((id) - 1) / m + 1)             // fid 逆变换：编号 id → 行号 x（1-indexed）
#define gy(id) (((id) - 1) % m + 1)             // fid 逆变换：编号 id → 列号 y（1-indexed）

// ============ 类型别名（与 ihxnan.hpp 完全一致，挤成一行省篇幅）============
using namespace std;using ll=long long;using ul=unsigned long long;using lll=__int128;using ld=long doub
le;using vb=vector<bool>;using vi=vector<int>;using vvi=vector<vi>;using vl=vector<ll>;using vvl=vector<
vl>;using pii=pair<int, int>;using pll=pair<ll,ll>;using ti=tuple<int,int,int>;using tl=tuple<ll,ll,ll>;

// ============ 常量 ============
const ll iINF=0x3f3f3f3f,mod=998244353,lINF=0x3f3f3f3f3f3f3f3f,MOD=1000000007;

#ifdef IHXNAN
    // gdb(a, b) 依次打印 "a = ... , b = ..."（变量名文本与值交替输出到 stderr）
    template <typename T> void _debug(char *f, T t) // 递归终点：只剩一个变量
    {
        cerr << f << " = " << t << endl;
    }
    template <typename T, typename... Ar> void _debug(char *f, T x, Ar... y) // 变参递归
    {
        while (*f != ',')           // 打印变量名部分（直到逗号）
            cerr << *f++;
        cerr << " = " << x << ",";  // 打印 " = 值,"
        _debug(f + 1, y...);        // 跳过逗号，递归处理剩余变量
    }
    #define gdb(...) _debug((char *)#__VA_ARGS__, __VA_ARGS__) // #__VA_ARGS__ 取变量名文本
#else
    #define gdb(...) void() // 提交模式：展开为空语句
#endif

void solve() // 由选手实现
{

}

int main()
{
    ios::sync_with_stdio(false); // 关闭与 C 标准库的同步，加速 cin/cout
    cin.tie(nullptr);            // 解除 cin 与 cout 绑定，避免每次读入前 flush
    cout.tie(nullptr);
    int t = 1;
    cin >> t;                    // 默认多组：无条件读入组数（单组题请注释掉本行）
```

```
    while (t--)
        solve();
    return 0;
}
```

Python 版骨架 `Template/ihxnan.py`

> 快速 IO 辅助函数：`I()` 读一行去空白、`II()` 读整数、`MII()` 读一行整数 map、`LII()` 读一行整数列表、`GMI()` / `LGMI()` 为 0-indexed 版（整体减一）。

```
import math
import sys
from collections import Counter, defaultdict, deque  # 常用容器
from heapq import heapify, heappop, heappush         # 堆（优先队列）

input = sys.stdin.readline       # 用 C 层 readline 替代 input，读整行更快
sys.setrecursionlimit(1000000)   # 抬高递归上限，避免深搜/递归爆栈


def I():                         # 读一行，去掉首尾空白（字符串）
    return input().strip()


def II():                        # 读一个整数（整行）
    return int(input())


def MII():                       # 读一行整数，返回 map（可 `a, b = MII()` 解包）
    return map(int, input().split())


def LII():                       # 读一行整数，返回 list
    return list(map(int, input().split()))


def GMI():                       # 同 MII，但整体减一（0-indexed 建图用）
    return map(lambda x: int(x) - 1, input().split())


def LGMI():                      # 同 LII，但整体减一
    return list(map(lambda x: int(x) - 1, input().split()))
```